

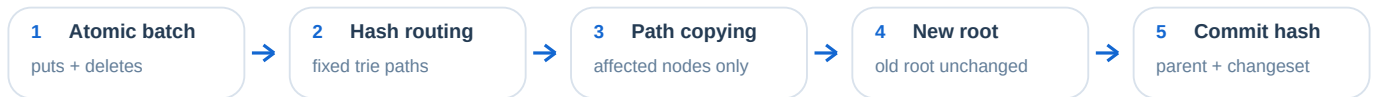
Incremental Merkle Trie

Verified core brief | Content-addressed structured-data versioning

Yes - the initial import builds once.

Later commits re-hash only changed leaves and their ancestor paths; unchanged subtrees retain their hashes.

INCREMENTAL COMMIT WORKFLOW



IMPLEMENTED AND TESTED

- Atomic multi-key **apply_changes()**
- Incremental **get / put / delete**
- Addressed nodes, changesets and commits
- Exact historical checkout
- Structured add / modify / delete diff
- Forced Log, forced Merkle and Hybrid modes
- Write, diff and exact-sharing instrumentation
- 25 automated tests, including randomized parity

VERIFIED CLI RUN

Dataset	1,000 rows
Mixed batch	3 updated + 1 added + 1 deleted
New trie nodes	19 (upper bound: 25)
Exact sharing	1,374 / 1,393 nodes = 98.64%
Default Hybrid	Log: 5 operations examined
Threshold = 2	Merkle: 87 node pairs compared

PASS: rebuild root, all diff modes, and both checkouts agree

RUN IT YOURSELF

POWERSHELL

```
cd C:\Users\admin\Desktop\Chronos
python chronos_cli_demo.py --show-changes
python chronos_cli_demo.py --hybrid-threshold 2
```

Interpretation. Version 1 is never overwritten. Version 2 is formed from new path nodes plus existing subtree hashes. The result is independently checked against a clean full rebuild. Timings printed by the CLI are local demonstration measurements, not final paper results.

Current boundary. In-memory store, linear HEAD-only history, and a provisional Hybrid threshold. Persistence, calibrated experiments, the Dolt adapter, backend API, and frontend remain next.

WHY THIS MODEL IS RESEARCHABLE

Commit scaling

Work follows the union of affected trie paths, not a full-state rebuild.

Two diff costs

Log cost follows intervening operations; Merkle cost follows unequal subtrees.

Chronos-H claim

Calibrate the crossover, then test whether adaptive selection stays near the faster mode.